

Workshop 2: Databases

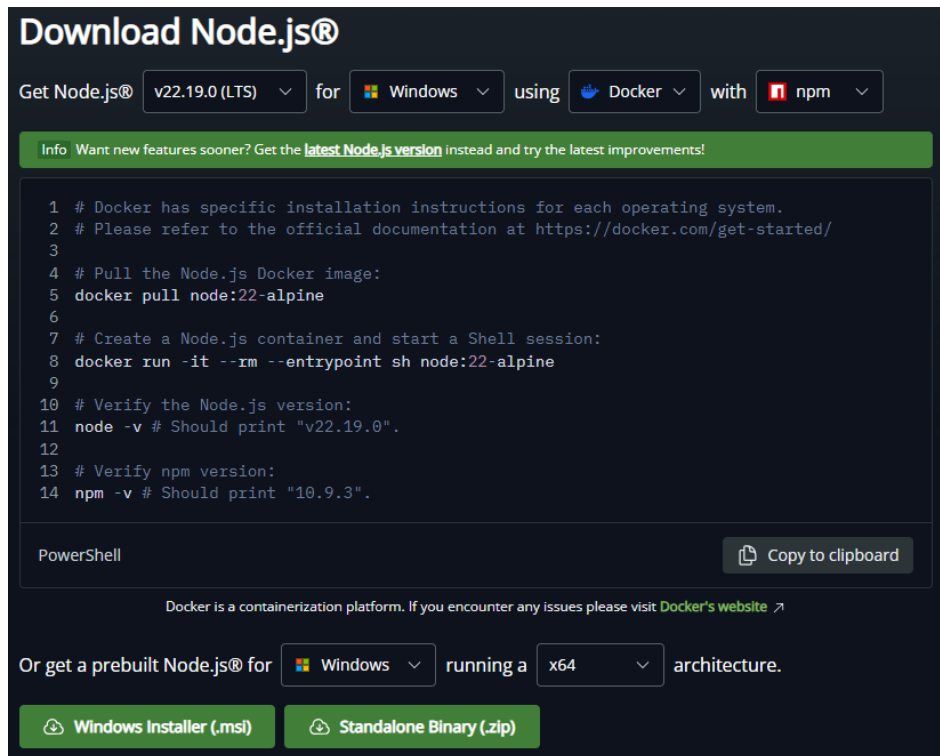
SQLite3 in Node.js with Express

This workshop is dedicated to Databases with a focus on SQLite3. You will work in a group of 4-5 students. You need one computer per group (of course, if you want to do the coding in parallel you can use more than one). The idea is to share your code within the group. You can create a shared text file with all the code you will run during this session. You **MUST** use the database you created on workshop 1 last week.

1) Node and Express - duration: 1,5 hour

5.a) Installing all the software...

Please, download and install Node.js on your computer.

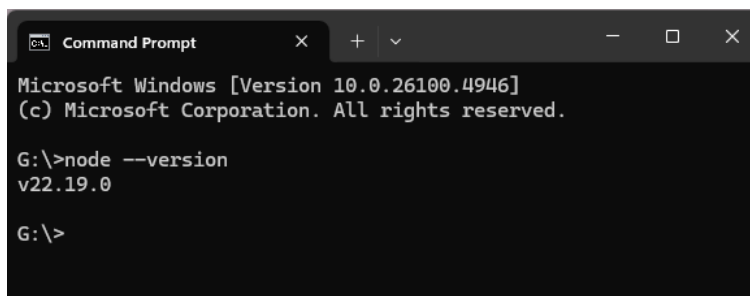


The screenshot shows the Node.js download page. At the top, it says "Download Node.js®". Below this, there are dropdown menus for "Get Node.js®" (set to v22.19.0 (LTS)), "for" (set to Windows), "using" (set to Docker), and "with" (set to npm). A green banner below these menus contains the text: "Info Want new features sooner? Get the latest Node.js version instead and try the latest improvements!". Below the banner is a code block with the following instructions:

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:22-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:22-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v22.19.0".
12
13 # Verify npm version:
14 npm -v # Should print "10.9.3".
```

Below the code block is a "PowerShell" label and a "Copy to clipboard" button. At the bottom, there is a note: "Docker is a containerization platform. If you encounter any issues please visit [Docker's website](#)". Below this, there are dropdown menus for "Or get a prebuilt Node.js® for" (set to Windows), "running a" (set to x64), and "architecture.". At the bottom, there are two buttons: "Windows Installer (.msi)" and "Standalone Binary (.zip)".

Open a command line interface (press the Windows key, then type CMD), type "node --version" (node space then dashdashversion with no space in between) to see if Node is correctly installed.



The screenshot shows a Windows Command Prompt window. The title bar says "Command Prompt". The window content shows the following text:

```
Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

G:\>node --version
v22.19.0

G:\>
```

If you obtain v22.19.0, it is perfect! You can go on; else, please ask for help...

Create a “db1” directory onto your Desktop (you will be able to move it afterwards at any other location). Go into this “db1” directory using the terminal and the “cd” command. When you will be in the right directory (“db1”), run the command “npm init -y”:

```
Command Prompt
Microsoft Windows [Version 10.0.19045.4780]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lanjer>cd Desktop

C:\Users\lanjer\Desktop>mkdir db1

C:\Users\lanjer\Desktop>cd db1

C:\Users\lanjer\Desktop\db1>npm init -y
Wrote to C:\Users\lanjer\Desktop\db1\package.json:

{
  "name": "db1",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

C:\Users\lanjer\Desktop\db1>
```

The previous command “npm init -y” created the file “package.json” in “db1”:

 package.json

The command name “npm” stands for Node Package Manager, it is a system to manage the packages and their dependencies when you need to add functionalities to your Node applications.

The “package.json” file contains important information about your Node project:

```
C: > Users > lanjer > Desktop > db1 > {} package.json > ...
1  {
2    "name": "db1",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": ""
12 }
```

Now, we need to install Express using the command: “npm install express”

```
C:\Users\lanjer\Desktop\db1>npm install express

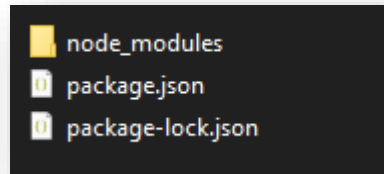
added 64 packages, and audited 65 packages in 2s

12 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\lanjer\Desktop\db1>_
```

This command should have added a “package-lock.json” file and a “node_modules” directory to your folder “db1”.



The directory “node_modules” contains the express package as well as all the other dependencies needed for it to work.

Let’s install a last package: “sqlite3” (guess how to do it and please, do it). You will get warning messages during this process (WARN) but it will not have any consequences for today.

We are ready to code our first Node/Express application using SQLite3! :)

5.b) Let’s code!

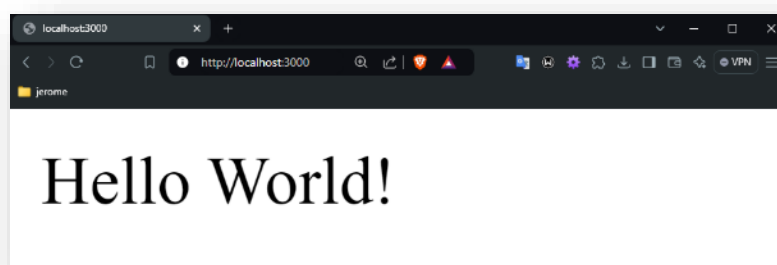
Using the text editor (Visual Studio Code please), create the “server.js” file in the “db1” directory and write the following Express code (you don’t need to write the green texts, it is only comments to help you understand the code):

```
JS server.js > ...
1  const express=require('express') // load the express package into the "express" variable
2  const app=express() // create the express application in the "app" variable
3  const port=3000 // define the port that will be used by the server
4
5  // define the default route "/"
6  app.get('/', (req, res) => {
7    |   res.send('Hello World!')
8  })
9
10 // listen for incoming requests onto the port and display a message into the console (terminal)
11 app.listen(port, () => {
12   |   console.log(`Example app listening on port ${port}...`) // be sure to use backticks on this line!
13 })
14
```

In the terminal, run this code: “node server.js”

```
C:\Users\lanjer\Desktop\db1>node server.js
Example app listening on port 3000...
```

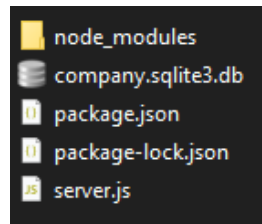
Open your browser and go to the page <http://localhost:3000>:



It works! Mycket bra!

But there is no database involved in the process. Let's add one...

At first, please copy the file "company.sqlite3.db" you have created into the "db1" directory:



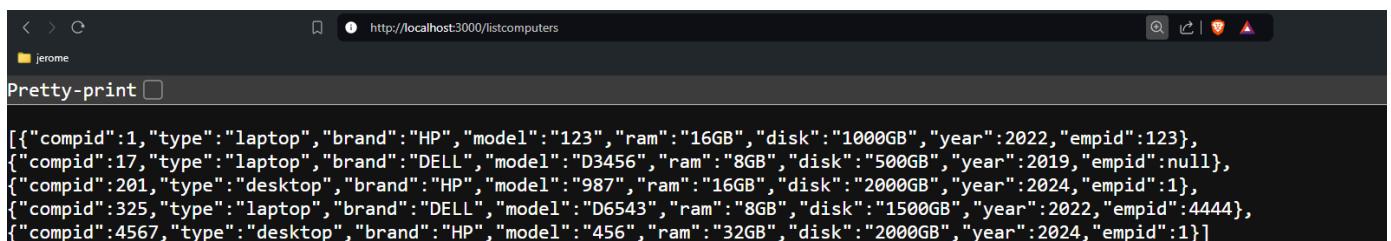
Modify the code to add a new route "/listcomputers" that will run the code to get the list of computers from the database.

```
1  const express=require('express') // load the express package into the "express" variable
2  const app=express() // create the express application in the "app" variable
3  const port=3000 // define the port that will be used by the server
4
5  // add the SQLite3 package to this server app
6  const sqlite3=require('sqlite3') // load the sqlite3 package into the "sqlite3" variable
7  const db=new sqlite3.Database('company.sqlite3.db') // load the database and put its content in the "db" variable
8
9  // define the default route "/"
10 app.get('/', (req, res) => {
11   res.send('Hello World!')
12 })
13
14 // define a new route "/listcomputers"
15 app.get('/listcomputers', (req, res) => {
16   db.all('SELECT * FROM Computer', (error, theComputers) => { // run a SQL query
17     if (error) { // if there is an error, send an error message
18       res.send('Sorry, there was an ERROR.')
19     } else { // no error, we can send back the result of the SQL query
20       res.send(theComputers) // send the RAW computers' list
21     }
22   })
23 })
24
25 // listen for incoming requests onto the port and display a message into the console (terminal)
26 app.listen(port, () => {
27   console.log(`Example app listening on port ${port}...`) // be sure to use backticks on this line!
28 })
29
```

Because you have changed the code, you NEED to stop the running node program by pressing CTRL+C and then, re-run the command node server.js in the terminal:

```
C:\Users\lanjer\Desktop\db1>node server.js
Example app listening on port 3000...
^C
C:\Users\lanjer\Desktop\db1>node server.js
Example app listening on port 3000...
```

Then, try the route: <http://localhost:3000/listcomputers> to see if it is working. You should obtain:



It works but we get all the results at the same time in a JSON variable. It is not user friendly.

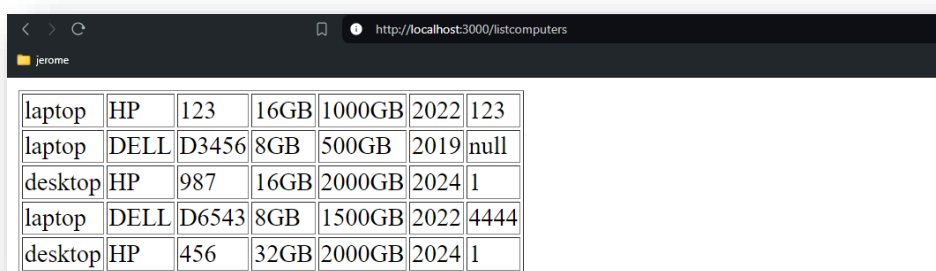
We want to get a more HTML styled result. To get one information at a time, **change** the “/listcomputers” route code to:

```
// define a new route "/liscomputers"
app.get('/listcomputers', (req, res) => {
  db.all('SELECT * FROM Computer', (error, theComputers) => { // run a SQL query
    if (error) { // if there is an error, send an error message
      res.send('Sorry, there was an ERROR.')
    } else { // no error, we can send back the result of the SQL query
      theHTMLCode='<table border="1px">'
      theComputers.forEach( (oneComputer) => {
        theHTMLCode+='<tr>' // normal ticks
        theHTMLCode+='<td>${oneComputer.type}</td>' // be careful backticks!
        theHTMLCode+='<td>${oneComputer.brand}</td>' // be careful backticks!
        theHTMLCode+='<td>${oneComputer.model}</td>' // be careful backticks!
        theHTMLCode+='<td>${oneComputer.ram}</td>' // be careful backticks!
        theHTMLCode+='<td>${oneComputer.disk}</td>' // be careful backticks!
        theHTMLCode+='<td>${oneComputer.year}</td>' // be careful backticks!
        theHTMLCode+='<td>${oneComputer.empid}</td>' // be careful backticks!
        theHTMLCode+='</tr>' // normal ticks
      })
      theHTMLCode+='</table>'
      res.send(theHTMLCode) // send the RAW computers' list
    }
  })
})
```

Because you have changed the code, you **NEED** to stop the running node program by pressing CTRL+C and then, re-run the command node server.js in the terminal:

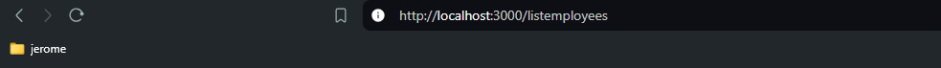
```
C:\Users\lanjer\Desktop\db1>node server.js
Example app listening on port 3000...
^C
C:\Users\lanjer\Desktop\db1>node server.js
Example app listening on port 3000...
```

You should get something like this (depending on the content of your Computer table in your database of course):



laptop	HP	123	16GB	1000GB	2022	123
laptop	DELL	D3456	8GB	500GB	2019	null
desktop	HP	987	16GB	2000GB	2024	1
laptop	DELL	D6543	8GB	1500GB	2022	4444
desktop	HP	456	32GB	2000GB	2024	1

Do the same thing for the “Employee” table by defining a new route “/listemployees” and the associated code.



1	Mira	POP	2000-01-02	0708091011	mira.pop@ju.se
2	Linus	RUDBECK	1999-03-04	0707070707	linus.rudbeck@ju.se
123	Jérôme	LANDRÉ	1972-10-11	0706050403	jerome.landre@ju.se
3210	Jasmin	JAKUPOVIC	1990-05-06	0711223344	jasmin.jakupovic@ju.se
4444	Anders	ARVIDSSON	1975-11-12	0701020304	anders.arvidsson@ju.se

OK, it was **not EASY**, I know.

“But using Node/Express/SQLite3 is the same as swimming or biking: once you know it, it is forever, it is always the same...”

That is why you **MUST go to the labs to ask questions** if you have problems with your code. The lab assistants will **help** you and give you **feedback** on your code! :)

TACK OCH GRATTIS FÖR IDAG!